Eduvos Assessment Coversheet

Academic Details

Campus: **Eduvos**

Faculty: **Information Technology**

Module Code: **ITNPA4-34**

Lecturer's Name: **Tinotenda Maveteta**

Group (if applicable) _____

Assessment Details

Assessment Type: **Project 1**

Assignment/Brief Number: _____

Due Date/ Submission Date: **6 November**

6Student Details (Group, if applicable)

#	Full Name	Student Number	Contact Number	% Participation	Signature
1					
2					
3					
4					
5					
6					

Student Details (Individual, if applicable)

Student Full Name: **Xikongomelo Max Mabunda**

Student Number: **EDUV6693976**

Academic Integrity

AI Disclosure Completed [X] **Yes** [ ] **No**

Declaration:
I declare that this assessment is my own original work except for source material explicitly acknowledged. I confirm that this work has not previously been submitted for assessments in any course. I acknowledge the institutional AI policy and confirm that AI tools have only been used as stipulated and documented in the AI Disclosure Appendix.

Student Signature 

Date **07 November 2026**

Lecturer Use Only

Lecturer Comments:

Marks Awarded (%):

Lecturer Signature:

Date:

Assessment

Question 1

1.1

```
import pandas as pd
import re

# load the data
df = pd.read_json("research_abstracts.json")

display(df.head())

print("\nDataset info")
df.info()

# check missing values
print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicats:")
print(df.duplicated().sum())

print("\nLabel counts:")
print(df["Label"].value_counts())

import nltk
from nltk.stem import WordNetLemmatizer

nltk.download("wordnet")

lemmatizer = WordNetLemmatizer()

# clean and reduce the words to their base form
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text)

    words = text.split()
    words = [lemmatizer.lemmatize(word) for word in words]

    return " ".join(words).strip()

df["Clean_Abstract"] = df["Abstract"].apply(clean_text)

print("\nOriginal and cleaned text:")
display(df[["Abstract", "Clean_Abstract", "Label"]].head())

print("\nShape of the dataset:", df.shape)
```

	DOI	Abstract	Label
0	10.1155/2021/1071145	Forward-looking forecasting of the inflation r...	ai
1	10.1007/s43546-022-00384-2	Our study examined the disaggregation of infla...	ai
2	10.1002/for.2667	This paper develops a dynamic factor model tha...	classical
3	10.1007/s42973-021-00091-x	In this paper, I use a simple SIR Macro model ...	classical
4	10.1007/s40258-012-0003-z	BackgroundGame theory is useful for identifyin...	classical

1.2

```
from sklearn.feature_extraction.text import TfidfVectorizer

# turn the cleaned abstracts into tf idf features
tfidf = TfidfVectorizer(stop_words="english")

X = tfidf.fit_transform(df["Clean_Abstract"])
y = df["Label"]

print("Shape of tf idF data:", X.shape)
print("Number of labels:", len(y))
```

```
Shape of tf idF data: (1138, 8268)
Number of labels: 1138
```

```
# check a few of the words used as features

print("\nSome of the features:")

print(tfidf.get_feature_names_out()[:20])
```

```
Some of the features:
['aa' 'aaa' 'abandoned' 'abandonment' 'abar' 'abbreviation' 'abct'
 'ability' 'able' 'abm' 'abnormal' 'abo' 'abortive' 'abroad' 'abrupt'
 'absence' 'absent' 'absenteeism' 'absolut' 'absolute']
```

```
from sklearn.model_selection import train_test_split

# split the data into training and testing sets
# stratify keeps the class proportions similar in both sets
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state=42,stratify=y)

print("train data:", X_train.shape)
print("Test Data:", X_test.shape)

# check that both sets still have the two classes
print("\nTraining labels:")
print(y_train.value_counts())

print("\nTesting labels:")

print(y_test.value_counts())
```

```
train data: (910, 8268)
Test Data: (228, 8268)

Training labels:
Label
classical    720
ai           190
Name: count, dtype: int64

Testing labels:
Label
classical    180
ai            48
Name: count, dtype: int64
```

1.3

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# build the baseline model
baseline_model = MultinomialNB()

baseline_model.fit(X_train, y_train)

# predictions for both training and testing data
y_train_pred = baseline_model.predict(X_train)
y_test_pred = baseline_model.predict(X_test)

# training performance
print("Training accuracy:", accuracy_score(y_train, y_train_pred))

print("\nTraining classification report:")
print(classification_report(y_train, y_train_pred))

print("\nTraining confusion matrix:")
print(confusion_matrix(y_train, y_train_pred))

# testing performance
print("\nTest accuracy:", accuracy_score(y_test, y_test_pred))

print("\nTest classification report:")
print(classification_report(y_test, y_test_pred))

print("\nTest confusion matrix:")
print(confusion_matrix(y_test, y_test_pred))
```

```
Training accuracy: 0.8098901098901099

Training classification report:
              precision    recall  f1-score   support

          ai       1.00      0.09      0.16       190
   classical       0.81      1.00      0.89       720

    accuracy                           0.81       910
   macro avg       0.90      0.54      0.53       910
weighted avg       0.85      0.81      0.74       910


Training confusion matrix:
[[ 17 173]
 [  0 720]]

Test accuracy: 0.793859649122807

Test classification report:
              precision    recall  f1-score   support

          ai       1.00      0.02      0.04        48
   classical       0.79      1.00      0.88       180

    accuracy                           0.79       228
   macro avg       0.90      0.51      0.46       228
weighted avg       0.84      0.79      0.71       228


Test confusion matrix:
[[  1  47]
 [  0 180]]
```

Question 2

2.1

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# create the four new models
logistic_model = LogisticRegression(max_iter=1000)
svm_model = LinearSVC()
knn_model = KNeighborsClassifier()
random_forest_model = RandomForestClassifier(random_state=42)

# train the models
logistic_model.fit(X_train, y_train)
svm_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)

print("All four models have been trained.")
```

```
# make predictions on the test data
logistic_pred = logistic_model.predict(X_test)
svm_pred = svm_model.predict(X_test)
knn_pred = knn_model.predict(X_test)
random_forest_pred = random_forest_model.predict(X_test)

# show the accuracy of each model
print("Logistic Regression accuracy:", accuracy_score(y_test, logistic_pred))
print("SVM accuracy:", accuracy_score(y_test, svm_pred))
print("KNN accuracy:", accuracy_score(y_test, knn_pred))
print("Random Forest accuracy:", accuracy_score(y_test, random_forest_pred))
```

```
Logistic Regression accuracy: 0.8464912280701754
SVM accuracy: 0.9035087719298246
KNN accuracy: 0.8728070175438597
Random Forest accuracy: 0.9035087719298246
```

```
print("Logistic Regression")
print(classification_report(y_test, logistic_pred))

print("SVM")
print(classification_report(y_test, svm_pred))

print("KNN")
print(classification_report(y_test, knn_pred))

print("Random Forest")
print(classification_report(y_test, random_forest_pred))
```

```
Logistic Regression
              precision    recall  f1-score   support

          ai       1.00      0.27      0.43        48
   classical       0.84      1.00      0.91       180

    accuracy                           0.85       228
   macro avg       0.92      0.64      0.67       228
weighted avg       0.87      0.85      0.81       228

SVM
              precision    recall  f1-score   support

          ai       0.93      0.58      0.72        48
   classical       0.90      0.99      0.94       180

    accuracy                           0.90       228
   macro avg       0.92      0.79      0.83       228
weighted avg       0.91      0.90      0.89       228

KNN
              precision    recall  f1-score   support

          ai       0.76      0.58      0.66        48
   classical       0.90      0.95      0.92       180

    accuracy                           0.87       228
   macro avg       0.83      0.77      0.79       228
weighted avg       0.87      0.87      0.87       228

Random Forest
              precision    recall  f1-score   support

          ai       1.00      0.54      0.70        48
   classical       0.89      1.00      0.94       180

    accuracy                           0.90       228
   macro avg       0.95      0.77      0.82       228
weighted avg       0.91      0.90      0.89       228
```

- SVM and Random Forest both achieved the highest accuracy at 90.35%.
- SVM had a slightly higher AI recall and AI F1-score than Random Forest.
- Logistic Regression and KNN performed below SVM and Random Forest. **Based on these results, SVM is the strongest model overall.**

2.2

SVM Performance Enhancement The c parameter controls the balance between correctly classifying the training data and allowing some classification errors. A higher c value makes the SVM place more importance on correctly classifying the training examples. For this enhancement, the c value was increased to 2 to determine whether it would improve the model's predictive performance.

```
# try a higher C value for the SVM
improved_svm = LinearSVC(C=2)
improved_svm.fit(X_train, y_train)
improved_svm_pred = improved_svm.predict(X_test)

print("Improved SVM accuracy:", accuracy_score(y_test, improved_svm_pred))
print("\nClassification report:")
print(classification_report(y_test, improved_svm_pred))
print("\nConfusion matrix:")
print(confusion_matrix(y_test, improved_svm_pred))
```

```
Improved SVM accuracy: 0.9122807017543859

Classification report:
              precision    recall  f1-score   support

          ai       0.94      0.62      0.75        48
   classical       0.91      0.99      0.95       180

    accuracy                           0.91       228
   macro avg       0.92      0.81      0.85       228
weighted avg       0.91      0.91      0.91       228


Confusion matrix:
[[ 30  18]
 [  2 178]]
```

2.3

```
import joblib

# save the trained SVM model
joblib.dump(improved_svm, "svm_model.pkl")

# save the TF-IDF vectorizer
joblib.dump(tfidf, "tfidf_vectorizer.pkl")
print("Model and vectorizer saved.")


%%writefile app.py

import streamlit as st
```

```
import joblib
import re
from nltk.stem import WordNetLemmatizer


# load the trained model and tf-idf vectorizer
model = joblib.load("svm_model.pkl")
tfidf = joblib.load("tfidf_vectorizer.pkl")

lemmatizer = WordNetLemmatizer()


# preprocess the new abstract
def clean_text(text):

    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text)

    words = text.split()
    words = [lemmatizer.lemmatize(word) for word in words]

    return " ".join(words).strip()


# page configuration
st.set_page_config(page_title="Researchly Research Abstract Classifier",page_icon=" 📄 ")


# title
st.title("Researchly Research Abstract Classifier")

st.write("This app classifies research abstracts as either "
    "AI-related or classical research.")


# abstract input
abstract = st.text_area("Enter research abstract:",height=200)

# classification
if st.button("Classify Abstract"):

    if abstract.strip():
        # preprocess the abstract
        cleaned_text = clean_text(abstract)

        # convert the abstract to tf-idf
        text_features = tfidf.transform([cleaned_text])

        # make the prediction
        prediction = model.predict(text_features)[0]

        # make the label lowercase
        prediction = str(prediction).lower()
```

```
        st.subheader("Prediction")

        if prediction == "ai":
            st.success("The abstract is classified as: artificial intelligence")

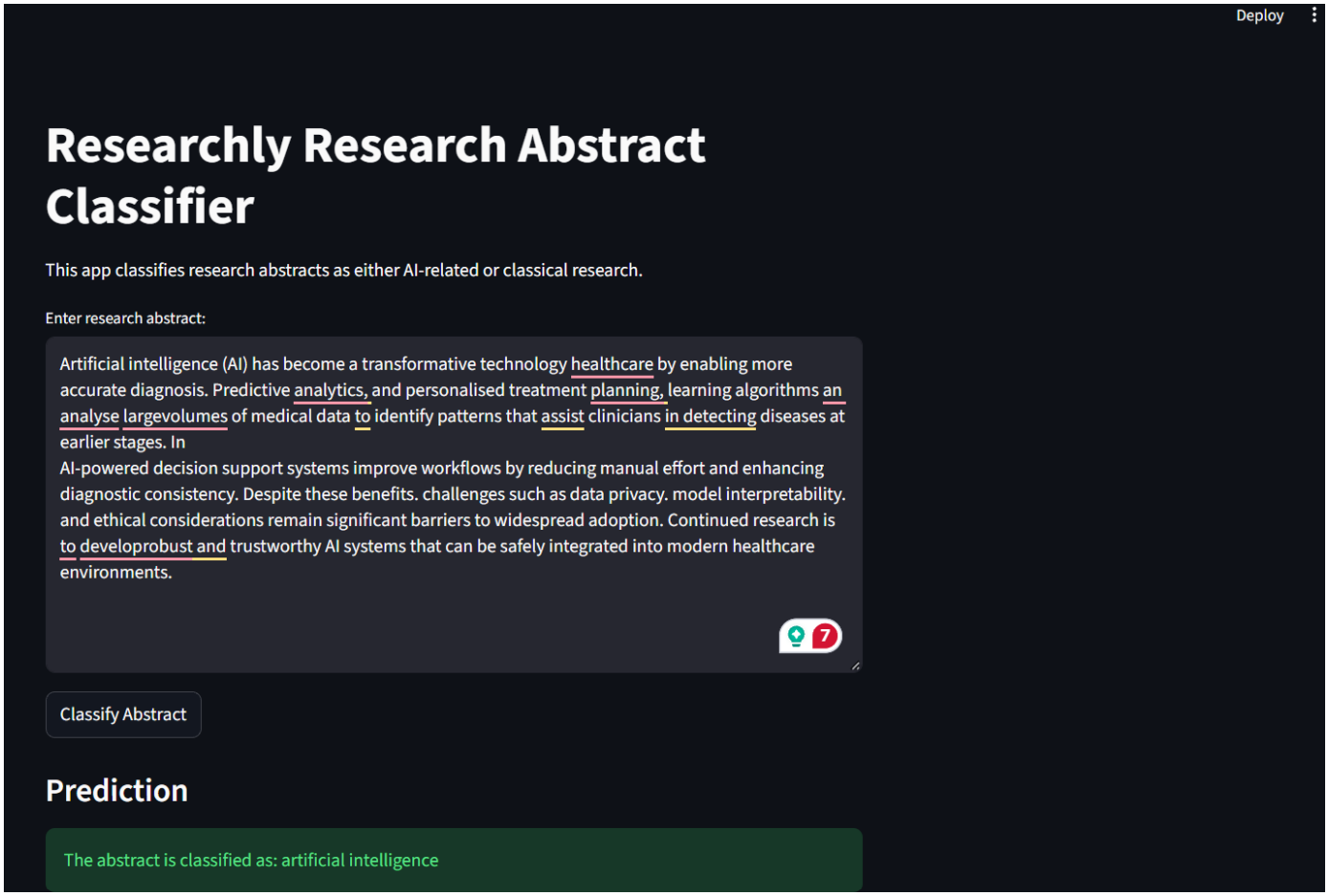
        else:
            st.success("The abstract is classified as: classical research")
    else:
        st.warning("Please enter a research abstract.")
```

```
import subprocess
import sys
import time

process = subprocess.Popen([sys.executable, "-m", "streamlit", "run","app.py", "--server.headless=true" ],stdout=subprocess.PIPE,stderr=subprocess.STDOUT,text=True)

time.sleep(3)

print("Streamlit application started.")
print("Open: http://localhost:8501")
```



Question 3

3.1

```
import pandas as pd
import glob
import os

# finding all research text files in the current folder.
# The * matches any characters between "research_" and ".txt".
files = sorted(glob.glob("research_corpus/research_*.txt"))

# read each article and store its filename and full content.
articles = []

for file in files:
    with open(file, "r", encoding="utf-8") as f:
        text = f.read()

    articles.append({"File": os.path.basename(file), "Article": text })

# Create a DataFrame containing all articles and their content.
df_articles = pd.DataFrame(articles)

# Display the filenames and confirm that all articles were loaded.
print("Number of articles:", len(df_articles))
display(df_articles[["File"]])

# Check that the full article content is stored.
display(df_articles[["File", "Article"]].head())

df_articles["Article"].str.len()
```

```
Number of articles: 5
```

	File
0	research_1.txt
1	research_2.txt
2	research_3.txt
3	research_4.txt
4	research_5.txt

	File	Article
0	research_1.txt	The purposes of this article are to position m...
1	research_2.txt	Now I see it, now I don't:\nresearcher's posit...
2	research_3.txt	DEVELOPMENTAL RESEARCH: STUDIES OF\nINSTRUCTIO...
3	research_4.txt	Abstract\nCronbach's alpha is a statistic comm...
4	research_5.txt	Denzin's Paradigm Shift: Revisiting Triangulat...

```
0    38160
1    16104
2    26154
3    38950
4    26831
Name: Article, dtype: int64
```

3.2

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# Download the language resources needed for preprocessing
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")

# Prepare the tools for cleaning the text
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

# Clean and preprocess an article
def denoise_text(text):
    # Convert text to lowercase
    text = text.lower()

    # Remove URLs and email addresses
    text = re.sub(r'http\S+|www\.\S+|\S+@\S+', ' ', text)

    # Remove apostrophes without splitting contractions into fragments
    text = re.sub(r"(?<=\w)[’](?=\w)", '', text)

    # Remove punctuation, numbers and unwanted characters
    text = re.sub(r'[^a-z\s]', ' ', text)

    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    # Tokenise the text into individual words
    words = text.split()

    # Remove stop words and lemmatise the remaining words
    words = [
        lemmatizer.lemmatize(word)
        for word in words
        if word not in stop_words]
    return ' '.join(words)

# Apply denoising to all five research articles
df_articles["Denoised_Article"] = df_articles["Article"].apply(denoise_text)

# Display the results
display(df_articles[["File", "Denoised_Article"]].head(5))
```

	File	Denoised_Article
0	research_1.txt	purpose article position mixed method research...
1	research_2.txt	see dont researcher position reflexivity quali...
2	research_3.txt	developmental research study instructional des...
3	research_4.txt	abstract cronbachs alpha statistic commonly qu...
4	research_5.txt	denzins paradigm shift revisiting triangulatio...

3.3

```
import nltk
from sumy.parsers.plaintext import PlaintextParser
from sumy.nlp.tokenizers import Tokenizer
from sumy.summarizers.text_rank import TextRankSummarizer

# Get the original text of the first research article
article_text = df_articles.loc[0, "Article"]

# Prepare the article for summarisation
parser = PlaintextParser.from_string(article_text, Tokenizer("english"))

# apply textRank summarisation
summarizer = TextRankSummarizer()
summary_sentences = summarizer(parser.document, 5)

# Display the summary
summary = " ".join(str(sentence) for sentence in summary_sentences)

print("Text rank summary — Research Article 1")
print(summary)
print("\nNumber of summary sentences:", len(summary_sentences))
```

```
Text rank summary — Research Article 1
The purposes of this article are to position mixed methods research (mixed research is a synonym) as the natural complement to traditional qualitative
and quantitative research, to present pragmatism as offering an attractive philosophical partner for mixed methods research, and to provide a framework
for designing and conducting mixed methods research. In doing this, we briefly review the paradigm “wars” and incompatibility thesis, we show some comm
onalities between quantitative and qualitative research, we explain the tenets of pragmatism, we explain the fundamental principle of mixed research an
d how to apply it, we provide specific sets of designs for the two major types of mixed methods research (mixed-model designs and mixed-method design
s), and, finally, we explain mixed methods research as following (recursively) an eight-step process. Basic agreement has been reached on each of the f
ollowing issues: (a) the relativity of the “light of reason” (i.e., what appears reasonable can vary across persons); (b) theory-laden perception or th
e theory-ladenness of facts (i.e., what we notice and observe is affected by our background knowledge, theories, and experiences; in short, observation
is not a perfect and direct window into “reality”); (c) underdetermination of theory by evidence (i.e., it is possible for more than one theory to fit
a single set of empirical data); (d) the DuhemQuine thesis or idea of auxiliary assumptions (i.e., a hypothesis cannot be fully tested in isolation bec
ause to make the test we also must make various assumptions; the hypothesis is embedded in a holistic network of beliefs; and alternative explanations
will continue to exist); (e) the problem of induction (i.e., the recognition that we only obtain probabilistic evidence, not final proof in empirical r
esearch; in short, we agree that the future may not resemble the past); (f) the social nature of the research enterprise (i.e., researchers are embedde
d in communities and they clearly have and are affected by their attitudes, values, and beliefs); and (g) the value-ladenness of inquiry (this is simil
ar to the last point but specifically points out that human beings can never be completely value free, and that values affect what we choose to investi
gate, what we see, and how we interpret what we see). In doing this, we briefly review the paradigm “wars” and incompatibility thesis, we show some com
monalities between quantitative and qualitative research, we explain the tenets of pragmatism, we explain the fundamental principle of mixed research a
nd how to apply it, we provide specific sets of designs for the two major types of mixed methods research (mixed-model designs and mixed-method design
s), and, finally, we explain mixed methods research as following (recursively) an eight-step process. Basic agreement has been reached on each of the f
ollowing issues: (a) the relativity of the “light of reason” (i.e., what appears reasonable can vary across persons); (b) theory-laden perception or th
e theory-ladenness of facts (i.e., what we notice and observe is affected by our background knowledge, theories, and experiences; in short, observation
is not a perfect and direct window into “reality”); (c) underdetermination of theory by evidence (i.e., it is possible for more than one theory to fit
a single set of empirical data); (d) the DuhemQuine thesis or idea of auxiliary assumptions (i.e., a hypothesis cannot be fully tested in isolation bec
ause to make the test we also must make various assumptions; the hypothesis is embedded in a holistic network of beliefs; and alternative explanations
will continue to exist); (e) the problem of induction (i.e., the recognition that we only obtain probabilistic evidence, not final proof in empirical r
esearch; in short, we agree that the future may not resemble the past); (f) the social nature of the research enterprise (i.e., researchers are embedde
d in communities and they clearly have and are affected by their attitudes, values, and beliefs); and (g) the value-ladenness of inquiry (this is simil
ar to the last point but specifically points out that human beings can never be completely value free, and that values affect what we choose to investi
gate, what we see, and how we interpret what we see).

Number of summary sentences: 5
```

```
import re
import matplotlib.pyplot as plt
from collections import Counter
from nltk.corpus import stopwords

# Remove common English words so the chart focuses on meaningful terms.
stop_words = set(stopwords.words("english"))

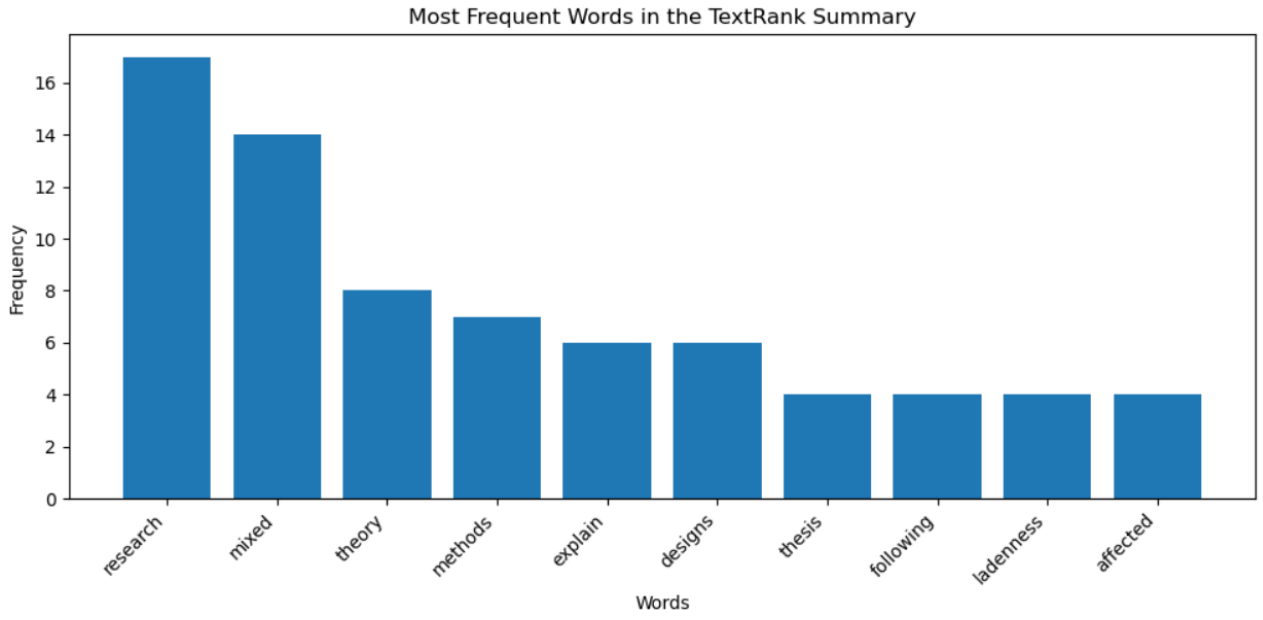
words = re.findall(r"\b[a-zA-Z]+\b", summary.lower())

summary_words = [a
    word for word in words
    if word not in stop_words and len(word) > 2
]

word_counts = Counter(summary_words).most_common(10)

# Display the most frequent words in a bar chart.
terms, counts = zip(*word_counts)

plt.figure(figsize=(10, 5))
plt.bar(terms, counts)
plt.title("Most Frequent Words in the TextRank Summary")
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



3.4

```
# Select the remaining four articles and combine their original text
remaining_articles = df_articles.loc[1:, "Article"]
combined_text = " ".join(remaining_articles)
print("Number of remaining articles:", len(remaining_articles))
print("Total words:", len(combined_text.split()))
```

```
Number of remaining articles: 4
Total words: 16193
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
import matplotlib.pyplot as plt

# treat each remaining article as a separate document.
vectorizer = TfidfVectorizer(stop_words="english",max_features=10)

tfidf_matrix = vectorizer.fit_transform(remaining_articles)

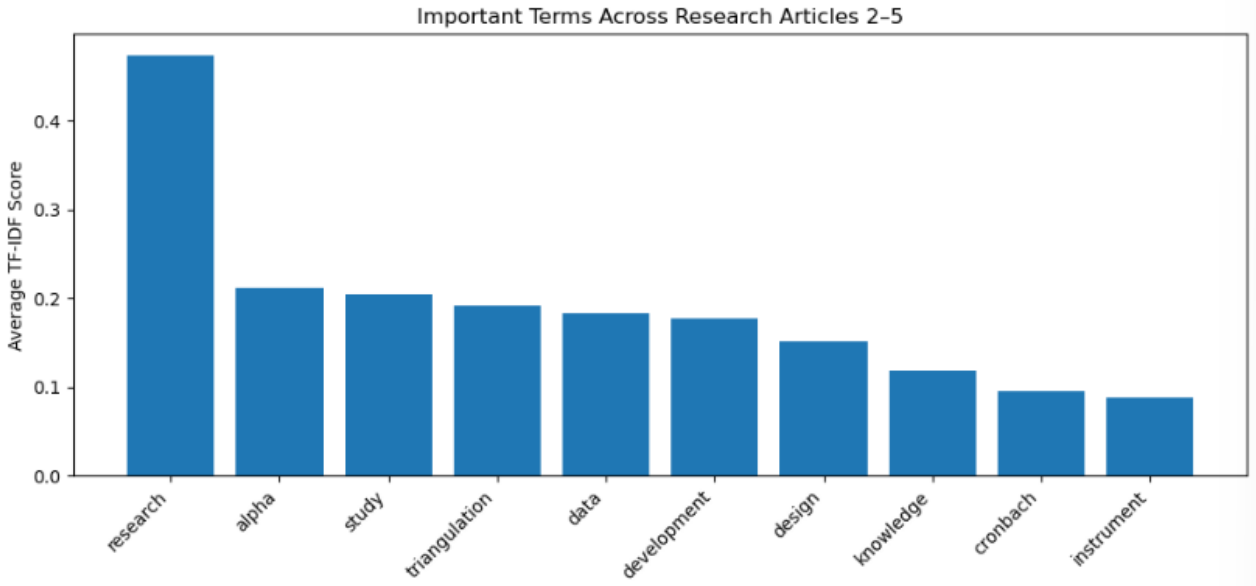
# Find the average tf-idf score of each term across the articles.
terms = vectorizer.get_feature_names_out()
scores = tfidf_matrix.mean(axis=0).A1

term_scores = pd.DataFrame({"Term": terms,"TF-IDF Score": scores}).sort_values("TF-IDF Score",
ascending=False)

display(term_scores)

# Visualise the terms with the highest average TF-IDF scores.
plt.figure(figsize=(10, 5))
plt.bar(term_scores["Term"], term_scores["TF-IDF Score"])
plt.title("Important Terms Across Research Articles 2–5")
plt.xlabel("Terms")
plt.ylabel("Average TF-IDF Score")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

	Term	TF-IDF Score
7	research	0.474424
0	alpha	0.211644
8	study	0.204907
9	triangulation	0.191766
2	data	0.183612
4	development	0.177396
3	design	0.151416
6	knowledge	0.118258
1	cronbach	0.095495
5	instrument	0.087978




```
from sumy.parsers.plaintext import PlaintextParser
from sumy.nlp.tokenizers import Tokenizer
from sumy.summarizers.lsa import LsaSummarizer

# Prepare the combined text of Articles 2–5.
parser = PlaintextParser.from_string(combined_text, Tokenizer("english"))

# using LSA to select important sentences.
summarizer_lsa = LsaSummarizer()
summary_sentences_lsa = summarizer_lsa(parser.document, 10)

# Combine and display the selected sentences.
collective_summary = " ".join(str(sentence) for sentence in summary_sentences_lsa)

print("LSA Summary Research articles 2–5")
print(collective_summary)
print("\nNumber of summary sentences:", len(summary_sentences_lsa))
```

```
LSA Summary Research articles 2-5
This article discusses issues relative to reflexivity under varying types of researcher’s personal experience with the world being studied and illust
rates them using my own work. Qualitative research primarily contributes to the development of “mirrors for man” so that we can see ourselves better
(Kluckhohn, as noted in Diesing, 1991). This conceptual framework not only has been commonly subscribed to, but was subsequently extended into the re
search, development, and diffusion model (Brickell, Clark & Guba, as cited in Havelock, 1971). We have learned from the research literature that tran
sfer of training is impacted by motivation, organizational climate, and previous educational experiences. Therefore, one may expand a front-end analy
sis to address such issues, or even construct design models that reflect this information, but the foundational research would not be considered deve
lopmental. Guidance is offered to authors reporting, and readers evaluating, studies that present Cronbach’s alpha statistic as evidence of instrumen
t quality. For example, one paper in the IJSE volume surveyed (Bathgate, Crowell, Schunn, Cannady, & Dorph, 2015) made two references to an unspecifi
ed alpha statistic. fairly high; not satisfactory overlapping with sufficient and acceptable) used across different scholars, so that terminology see
ms to be somewhat arbitrary. Likewise, Denzin (1978) argued that researchers bring their personal beliefs in addition to the social and political env
ironment, which eliminates any possibility of conducting value-free research. A case study design with at least two data collection methods (Amerson,
2011; Houghton, Casey, Shaw, & Murphy, 2013) is one example of methodological triangulation.

Number of summary sentences: 10
```

```
# Display the LSA summary as separate sentences for inspection.
for i, sentence in enumerate(summary_sentences_lsa, start=1):
    print(f"{i}. {sentence}\n")
```

```
1. This article discusses issues relative to reflexivity under varying types of researcher’s personal experience with the world being studied and illus
trates them using my own work.

2. Qualitative research primarily contributes to the development of “mirrors for man” so that we can see ourselves better (Kluckhohn, as noted in Diesi
ng, 1991).

3. This conceptual framework not only has been commonly subscribed to, but was subsequently extended into the research, development, and diffusion mode
l (Brickell, Clark & Guba, as cited in Havelock, 1971).

4. We have learned from the research literature that transfer of training is impacted by motivation, organizational climate, and previous educational e
xperiences.

5. Therefore, one may expand a front-end analysis to address such issues, or even construct design models that reflect this information, but the founda
tional research would not be considered developmental.

6. Guidance is offered to authors reporting, and readers evaluating, studies that present Cronbach’s alpha statistic as evidence of instrument quality.

7. For example, one paper in the IJSE volume surveyed (Bathgate, Crowell, Schunn, Cannady, & Dorph, 2015) made two references to an unspecified alpha s
tatistic.

8. fairly high; not satisfactory overlapping with sufficient and acceptable) used across different scholars, so that terminology seems to be somewhat a
rbitrary.

9. Likewise, Denzin (1978) argued that researchers bring their personal beliefs in addition to the social and political environment, which eliminates a
ny possibility of conducting value-free research.

10. A case study design with at least two data collection methods (Amerson, 2011; Houghton, Casey, Shaw, & Murphy, 2013) is one example of methodologic
al triangulation.
```

```
# Generate a longer LSA summary so we have more sentences to choose from.
summary_sentences_lsa = summarizer_lsa(parser.document, 15)

# Remove very short sentence fragments and duplicate sentences.
clean_summary_sentences = []
seen_sentences = set()

for sentence in summary_sentences_lsa:
    sentence_text = str(sentence).strip()
    sentence_key = sentence_text.lower()

    if len(sentence_text.split()) >= 8 and sentence_key not in seen_sentences:
        clean_summary_sentences.append(sentence_text)
        seen_sentences.add(sentence_key)

collective_summary = " ".join(clean_summary_sentences)

print("Improved LSA Summary Research Articles 2–5\n")
print(collective_summary)
print("\nNumber of selected sentences:", len(clean_summary_sentences))
```

```
Improved LSA Summary Research Articles 2–5

This article discusses issues relative to reflexivity under varying types of researcher’s personal experience with the world being studied and illustra
tes them using my own work. Qualitative research primarily contributes to the development of “mirrors for man” so that we can see ourselves better (Klu
ckhohn, as noted in Diesing, 1991). This conceptual framework not only has been commonly subscribed to, but was subsequently extended into the researc
h, development, and diffusion model (Brickell, Clark & Guba, as cited in Havelock, 1971). Such activities “could change the development of products int
o research with empirical results and theory generalizable to new problems” (p. 148). We have learned from the research literature that transfer of tra
ining is impacted by motivation, organizational climate, and previous educational experiences. Therefore, one may expand a front-end analysis to addres
s such issues, or even construct design models that reflect this information, but the foundational research would not be considered developmental. Guid
ance is offered to authors reporting, and readers evaluating, studies that present Cronbach’s alpha statistic as evidence of instrument quality. For ex
ample, one paper in the IJSE volume surveyed (Bathgate, Crowell, Schunn, Cannady, & Dorph, 2015) made two references to an unspecified alpha statistic.
Two other papers (that did not use the terms reliability or internal consistency) did specify that alpha was concerned with items being on a single sca
le or unidimensionality (as will be discussed later in this article, Cronbach’s alpha does not provide evidence of scale unidimensionality.) fairly hig
h; not satisfactory overlapping with sufficient and acceptable) used across different scholars, so that terminology seems to be somewhat arbitrary. A s
omewhat similar use was made of Cronbach alpha reliabilities by Arslan, Cigdemoglu and Moseley (2012), who report a “three-tier diagnostic test to asse
ss pre-service teachers’ misconceptions about global warming, greenhouse effect, ozone layer depletion, and acid rain” through the development of the a
tmosphere-related environmental problems diagnostic test (AREPDiT). The intended audience is novice researchers and doctoral students who can benefit f
rom clarity on some common misconceptions about triangulation. Likewise, Denzin (1978) argued that researchers bring their personal beliefs in addition
to the social and political environment, which eliminates any possibility of conducting value-free research. Moreover, researchers often use a qualitat
ive methodology to address social change, but struggle with concepts of objectivity, truth, and validity. A case study design with at least two data co
llection methods (Amerson, 2011; Houghton, Casey, Shaw, & Murphy, 2013) is one example of methodological triangulation.

Number of selected sentences: 15
```

```
import re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

# Remove the incomplete alpha statistic fragment from the summary.
clean_collective_summary = re.sub( r"fairly high; not satisfactory.*?somewhat arbitrary\.", "",
collective_summary,  flags=re.IGNORECASE).strip()

# Display the cleaned collective summary.
print("Collective Summary of Research Articles 2–5\n")
print(clean_collective_summary)

# Create a word cloud from the collective summary.
wordcloud = WordCloud( width=1000, height=500, background_color="white",
stopwords=set(stop_words), max_words=50).generate(clean_collective_summary)

plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Word Cloud of the Collective LSA Summary")
plt.show()
```

```
Collective Summary – Research Articles 2–5

This article discusses issues relative to reflexivity under varying types of researcher's personal experience with the world being studied and illustra
tes them using my own work. Qualitative research primarily contributes to the development of "mirrors for man" so that we can see ourselves better (Klu
ckhohn, as noted in Diesing, 1991). This conceptual framework not only has been commonly subscribed to, but was subsequently extended into the researc
h, development, and diffusion model (Brickell, Clark & Guba, as cited in Havelock, 1971). Such activities "could change the development of products int
o research with empirical results and theory generalizable to new problems" (p. 148). We have learned from the research literature that transfer of tra
ining is impacted by motivation, organizational climate, and previous educational experiences. Therefore, one may expand a front-end analysis to addres
s such issues, or even construct design models that reflect this information, but the foundational research would not be considered developmental. Guid
ance is offered to authors reporting, and readers evaluating, studies that present Cronbach's alpha statistic as evidence of instrument quality. For ex
ample, one paper in the IJSE volume surveyed (Bathgate, Crowell, Schunn, Cannady, & Dorph, 2015) made two references to an unspecified alpha statistic.
Two other papers (that did not use the terms reliability or internal consistency) did specify that alpha was concerned with items being on a single sca
le or unidimensionality (as will be discussed later in this article, Cronbach's alpha does not provide evidence of scale unidimensionality.)  A somewha
t similar use was made of Cronbach alpha reliabilities by Arslan, Cigdemoglu and Moseley (2012), who report a "three-tier diagnostic test to assess pre
-service teachers' misconceptions about global warming, greenhouse effect, ozone layer depletion, and acid rain" through the development of the atmosph
ere-related environmental problems diagnostic test (AREPDiT). The intended audience is novice researchers and doctoral students who can benefit from cl
arity on some common misconceptions about triangulation. Likewise, Denzin (1978) argued that researchers bring their personal beliefs in addition to th
e social and political environment, which eliminates any possibility of conducting value-free research. Moreover, researchers often use a qualitative m
ethodology to address social change, but struggle with concepts of objectivity, truth, and validity. A case study design with at least two data collect
ion methods (Amerson, 2011; Houghton, Casey, Shaw, & Murphy, 2013) is one example of methodological triangulation.
```

Word Cloud of the Collective LSA Summary



Question 4

```
import pandas as pd
import glob
import os

# Locate the research article text files
article_files = sorted(glob.glob("research_corpus/research_*.txt"))

# Load the articles into a DataFrame
articles = []

for file in article_files:
    with open(file, "r", encoding="utf-8") as f:
        articles.append({
            "File": os.path.basename(file),
            "Article": f.read()})

df_articles = pd.DataFrame(articles)

# Check the loaded articles
print("Number of articles:", len(df_articles))
display(df_articles[["File"]])
```

```
Number of articles: 5
```

	File
0	research_1.txt
1	research_2.txt
2	research_3.txt
3	research_4.txt
4	research_5.txt

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer

# downloading  NLTK resources
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")

# Set up text-processing tools
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

# Clean and preprocess each article
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    words = text.split()

    words = [
        lemmatizer.lemmatize(word)
        for word in words
        if word not in stop_words and len(word) > 2]

    return " ".join(words)

df_articles["Processed_Article"] = df_articles["Article"].apply(preprocess_text)

# Create a filtered document-term matrix
vectorizer = CountVectorizer(max_df=0.80,min_df=2)

dtm = vectorizer.fit_transform(df_articles["Processed_Article"])

print("Document-term matrix shape:", dtm.shape)
print("Number of documents:", dtm.shape[0])
print("Number of unique terms:", len(vectorizer.get_feature_names_out()))
```

```
Document-term matrix shape: (5, 781)
Number of documents: 5
Number of unique terms: 781
```

```
from sklearn.decomposition import LatentDirichletAllocation

# Set the number of topics
```

```
n_topics = 3

# Train LDA using the updated document-term matrix
lda_model =
LatentDirichletAllocation(n_components=n_topics,random_state=42,max_iter=20,learning_method="batch
")

lda_model.fit(dtm)
print("LDA model trained successfully.")
print("Number of topics:", lda_model.n_components)
```

```
LDA model trained successfully.
Number of topics: 3
```

```
# Get the updated vocabulary
feature_names = vectorizer.get_feature_names_out()

# Display the top 10 words for each topic
n_top_words = 10

for topic_idx, topic in enumerate(lda_model.components_):
    top_word_indices = topic.argsort()[-n_top_words:][::-1]
    top_words = [feature_names[i] for i in top_word_indices]

    print(f"\nTopic {topic_idx + 1}:")
    print(", ".join(top_words))
```

```
Topic 1:
alpha, triangulation, instrument, cronbach, reliability, science, item, student, author, education

Topic 2:
development, design, evaluation, model, product, orientation, term, system, specific, even

Topic 3:
method, mixed, quantitative, design, paradigm, multiple, reality, position, important, approach
```

```
# Calculate topic probabilities using the updated LDA model
topic_distribution = lda_model.transform(dtm)

topic_columns = [f"Topic {i + 1}" for i in range(n_topics)]

df_topic_distribution = pd.DataFrame(topic_distribution * 100,columns=topic_columns)

df_topic_distribution.insert(0, "Article", df_articles["File"])
df_topic_distribution[topic_columns] = (df_topic_distribution[topic_columns].round(2))
display(df_topic_distribution)
```

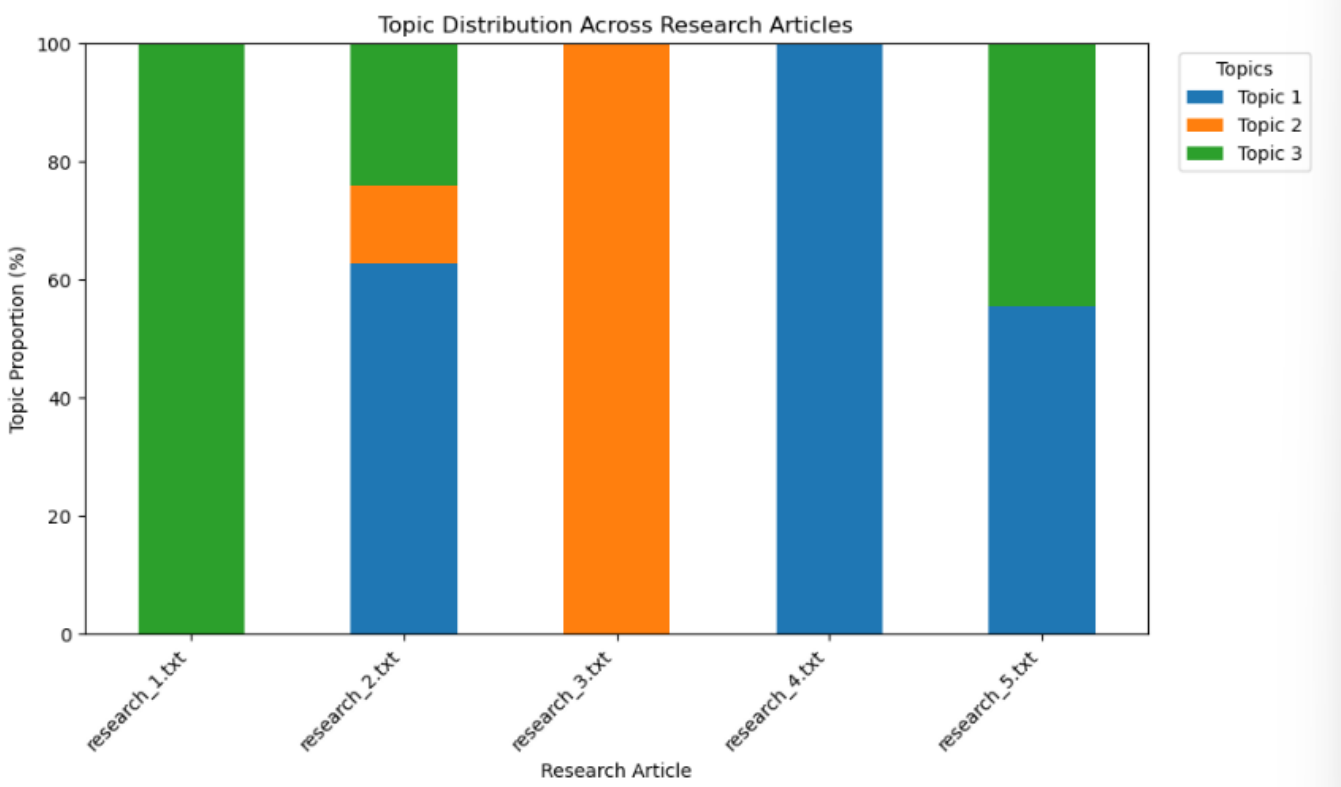
	Article	Topic 1	Topic 2	Topic 3
0	research_1.txt	0.02	0.02	99.96
1	research_2.txt	62.80	13.17	24.03
2	research_3.txt	0.03	99.93	0.03
3	research_4.txt	99.96	0.02	0.02
4	research_5.txt	55.52	0.03	44.45

```
import matplotlib.pyplot as plt

# Prepare the updated topic distribution data
plot_data = df_topic_distribution.set_index("Article")

# Plot topic proportions for each article
plot_data.plot(kind="bar",stacked=True,figsize=(10, 6))

plt.title("Topic Distribution Across Research Articles")
plt.xlabel("Research Article")
plt.ylabel("Topic Proportion (%)")
plt.xticks(rotation=45, ha="right")
plt.ylim(0, 100)
plt.legend(title="Topics", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



Interpretation of Results

- Research 1 is mainly associated with Topic 3 (99.96%).
- Research 2 has a higher proportion of Topic 1 (62.80%), with Topic 3 accounting for 24.03% and Topic 2 for 13.17%.
- Research 3 is mostly associated with Topic 2 (99.93%).
- Research 4 is almost entirely associated with Topic 1 (99.96%).
- Research 5 contains a mixture of Topic 1 (55.52%) and Topic 3 (44.45%).
- Overall, the articles show different topic distributions. While some are strongly linked to one topic, others contain a mix of topics. The top words for each topic can help explain what these topics represent. Since the analysis uses only five articles, the results should be interpreted with caution.

```
import matplotlib.pyplot as plt

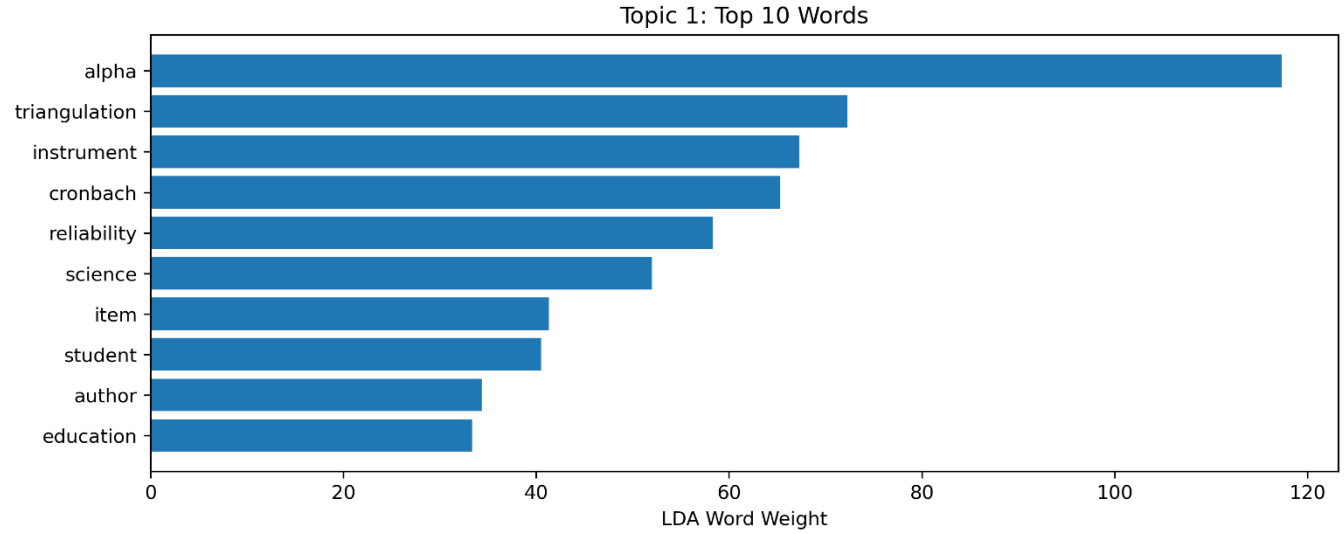
feature_names = vectorizer.get_feature_names_out()
n_top_words = 10

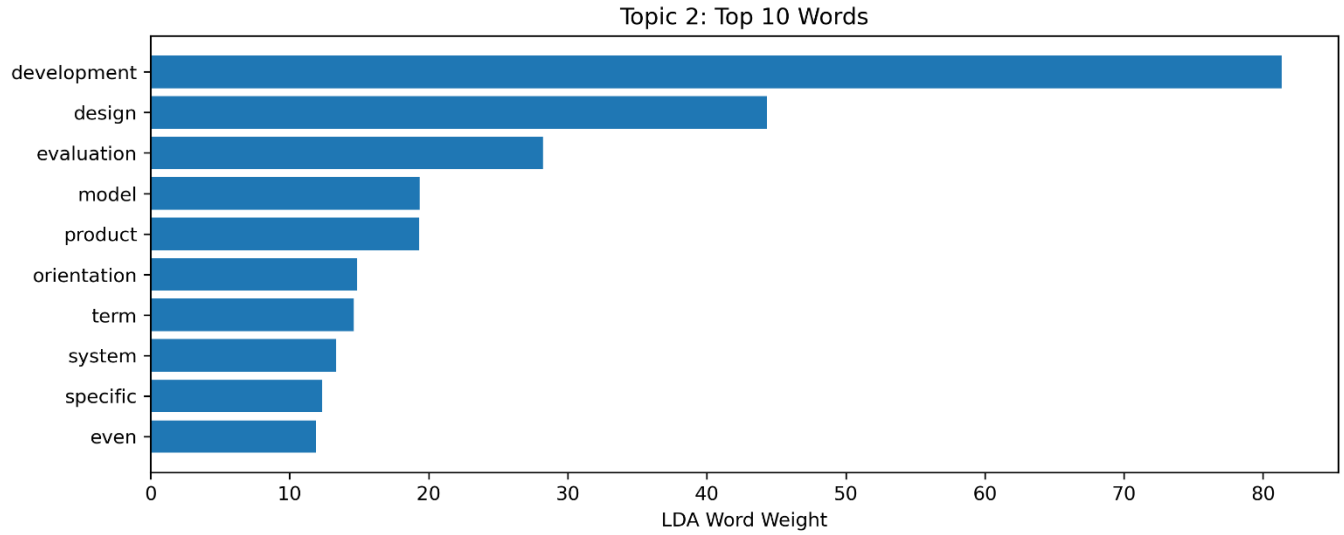
fig, axes = plt.subplots(nrows=n_topics,ncols=1,figsize=(10, 12))
```

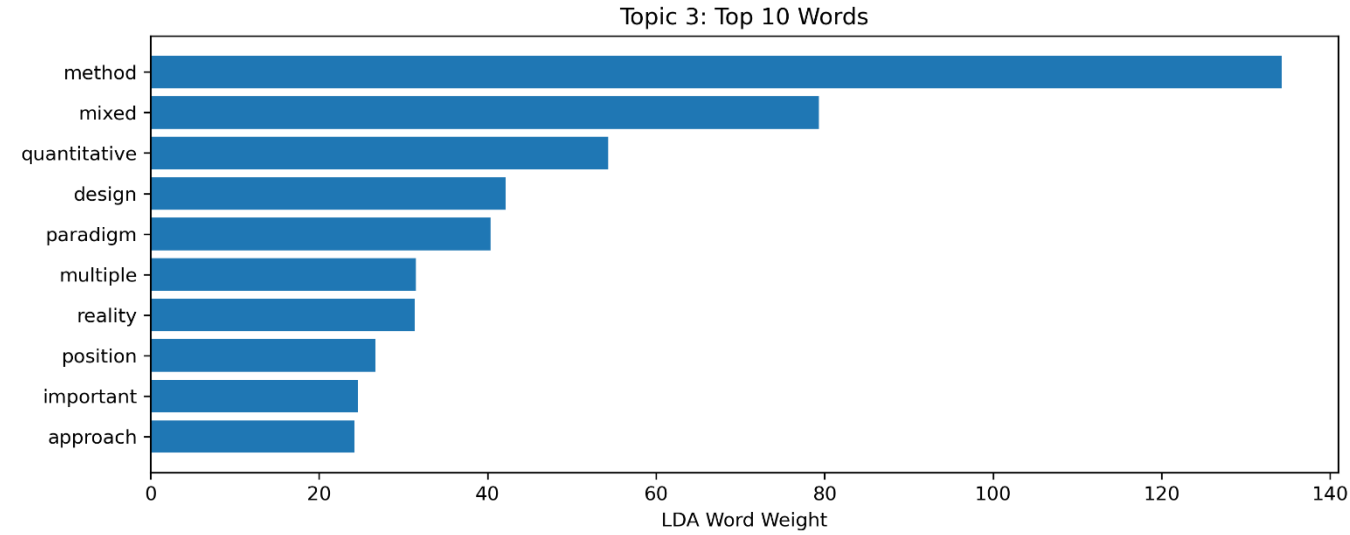
```
for topic_idx, topic in enumerate(lda_model.components_):
    top_indices = topic.argsort()[-n_top_words:]
    top_words = feature_names[top_indices]
    top_weights = topic[top_indices]

    axes[topic_idx].barh(top_words, top_weights)
    axes[topic_idx].set_title(f"Topic {topic_idx + 1}: Top 10 Words")
    axes[topic_idx].set_xlabel("LDA Word Weight")

plt.tight_layout()
plt.show()
```







Interpretation of Topic Words

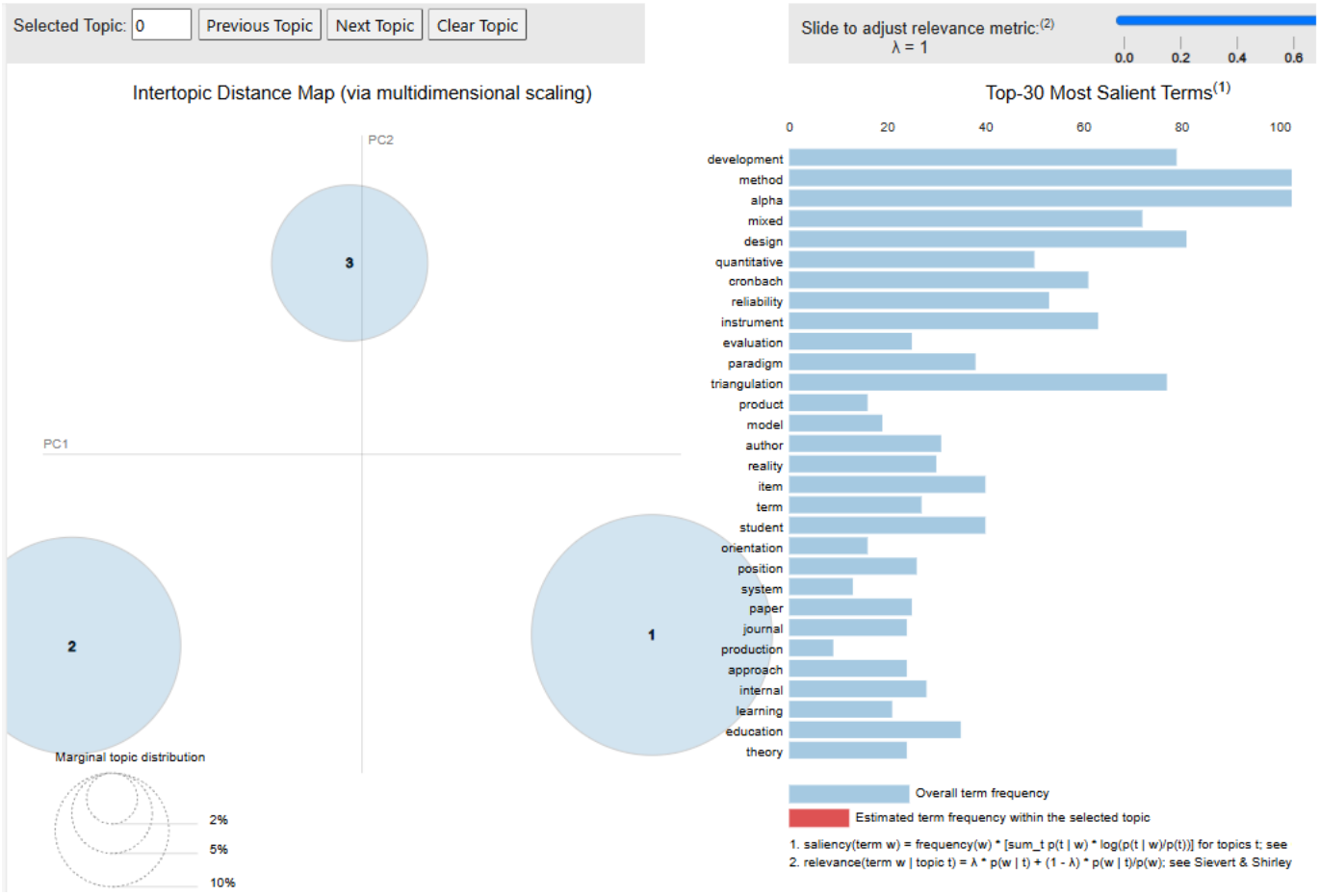
- **Topic 1:** The words relate mainly to reliability, measurement instruments and triangulation, with terms such as Cronbach, reliability and instrument.
- **Topic 2:** The main words are *development*, *design* and *evaluation*, suggesting a focus on developing and evaluating models or systems.
- **Topic 3:** Words such as method, mixed, quantitative and paradigm suggest a focus on research methods and approaches.

The topics are based on the words identified by the LDA model. Some words may overlap between topics, so the original articles should also be considered when interpreting them.

4.2

```
import pyLDAvis
import pyLDAvis.lda_model

# Prepare the interactive LDA visualisation
vis = pyLDAvis.lda_model.prepare(lda_model,dtm,vectorizer,mds="pcoa")
pyLDAvis.display(vis)
```



AI Disclosure Appendix

AI tool used	Purpose / how it was used	How I verified or adapted the output
ChatGPT 4o	Used to debug code if I could not understand the error	Ran the code to see if it worked, but only the parts that had errors; redid the code that relates to the task.
Grammarly	Checking concordant agreement for the interpretations	Used it in moderation to still articulate what I was expressing.

Declaration:

I/we declare that this assessment is my/our own original work, except for source material explicitly acknowledged. I/we confirm that any use of Artificial Intelligence (AI) tools has been applied only as permitted by the assessment instructions and has been fully disclosed where required.

I/we understand that I/we remain responsible for the accuracy, originality, and academic quality of this submission. I/we am/are aware of the institution's policies on academic integrity and the requirements relating to the responsible use of AI in assessments.

I/we acknowledge that failure to comply with these requirements, including undeclared or inappropriate AI use, may result in academic penalties or disciplinary procedures in accordance with institutional policy.

Student Signature(s)	Date
	**6 November 2026**